

ft_irc

Error・Reply詳細設計書

Numeric Reply・通知形式・異常処理

設計対象: Mandatory要件のみ

要件参照元: ft_irc subject Version 11.0

文書番号: 06

文書言語: 日本語

目次

1. 文書の目的	3	12. 入力Validation Error	14
2. IRC送信行の共通形式	4	13. 512 byte制限	15
3. Prefix	5	14. system call Error分類	16
4. Numeric Reply形式	6	15. 内部Errorと不変条件違反	17
5. Command通知形式	7	16. Buffer Error	18
6. Welcome Reply	8	17. Reply生成の安全性	19
7. Channel成功Reply	9	18. Error発生時の状態変更規則	20
8. 必須Error Numeric一覧	10	19. Replyテスト表	21
9. Numeric定数	11	20. 参照資料	22
10. Error Reply Helper	12	21. 実装完了条件	23
11. CommandとErrorの対応	13		

1. 文書の目的

本書は、IRC送信文字列、Prefix、Numeric Reply、Command通知、入力エラー、system callエラー、内部不整合への対応を実装するための詳細設計を定義する。

目標:

- ・ 返信形式をHandlerから分離する
- ・ 同じErrorを同じ形式で返す
- ・ 未登録Clientにも安全にReplyする
- ・ Error発生時に状態を途中変更しない
- ・ 1Clientの異常でServer全体を停止しない
- ・ 致命的エラーと接続単位エラーを区別する

2. IRC送信行の共通形式

Serverから送信する行:

```
[:<prefix> ]<command> [parameters]\r\n
```

規則:

- ・ Prefixがある場合は行頭のcolonから始める
- ・ PrefixとCommandの間はASCII space 1個
- ・ Parameter間はASCII space 1個
- ・ spaceを含む最後のParameterはcolon付きTrailingにする
- ・ 行末は必ずCRLF
- ・ CRLFを含む全長を512 byte以下にする

ReplyはCRLFなしの本文を生成し、Server::queueToClient()がCRLFを付与する。

3. Prefix

Server Prefix:

```
:ircserv.local
```

Client Prefix:

```
:<nickname>!<username>@<hostname>
```

例:

```
:alice!a@127.0.0.1
```

未設定項目のfallback:

項目	fallback
Nickname	*
Username	unknown
Hostname	0.0.0.0

Client通知は登録後のClientについてのみ生成するため、通常fallbackは使用されない。

4. Numeric Reply形式

```
:<serverName> <3-digit-code> <target> <parameters>
```

target:

- ・ Nickname設定済みならNickname
- ・ 未設定なら*

例:

```
:ircserv.local 433 * alice :Nickname is already in use
```

生成API:

```
std::string Reply::numeric(  
    const std::string &serverName,  
    const Client &target,  
    int code,  
    const std::string &parameters  
);
```

3桁変換:

```
std::ostringstream oss;  
oss << std::setw(3) << std::setfill('0') << code;
```

C++98の<iomanip>を使用する。

5. Command通知形式

生成API:

```
std::string Reply::command(  
    const std::string &prefix,  
    const std::string &command,  
    const std::string &parameters  
);
```

例:

```
:alice!a@127.0.0.1 JOIN :#general  
:alice!a@127.0.0.1 PRIVMSG #general :hello  
:alice!a@127.0.0.1 MODE #general +o bob
```

prefix引数は先頭colonなしで渡し、Reply側がcolonを付ける。

6. Welcome Reply

登録完了時に次の順序で送る。

001 RPL_WELCOME

```
:ircserv.local 001 <nick> :Welcome to the Internet Relay Network <nick>!<user>@<host>
```

002 RPL_YOURHOST

```
:ircserv.local 002 <nick> :Your host is ircserv.local, running version 1.0
```

003 RPL_CREATED

```
:ircserv.local 003 <nick> :This server was created <server-start-time>
```

Server開始時刻はConstructorで文字列化して保存する。C++標準ライブラリのstd::time()とstd::strftime()を使用する。

004 RPL_MYINFO

```
:ircserv.local 004 <nick> ircserv.local 1.0 - itkol
```

User Modeは実装しないため-とする。

422 ERR_NOMOTD

```
:ircserv.local 422 <nick> :MOTD File is missing
```

MOTDを実装しないことをClientへ明示し、登録シーケンスを終了させる。

7. Channel成功Reply

324 RPL_CHANNELMODEIS

```
:ircserv.local 324 <nick> <channel> +<modes> [mode parameters]
```

331 RPL_NOTOPIC

```
:ircserv.local 331 <nick> <channel> :No topic is set
```

332 RPL_TOPIC

```
:ircserv.local 332 <nick> <channel> :<topic>
```

341 RPL_INVITING

```
:ircserv.local 341 <nick> <channel> <targetNick>
```

353 RPL_NAMREPLY

```
:ircserv.local 353 <nick> = <channel> :@operator user
```

366 RPL_ENDOFNAMES

```
:ircserv.local 366 <nick> <channel> :End of NAMES list
```

Names Replyが512 byteを超える場合:

- ・ Nickname一覧を複数の353へ分割する
- ・ 各353を512 byte以内にする
- ・ 最後に366を1回送る

8. 必須Error Numeric一覧

Code	Symbol	Parameters
401	ERR_NOSUCHNICK	<nick> :No such nick/channel
403	ERR_NOSUCHCHANNEL	<channel> :No such channel
404	ERR_CANNOTSENDTOCHAN	<channel> :Cannot send to channel
409	ERR_NOORIGIN	:No origin specified
411	ERR_NORECIPIENT	:No recipient given (PRIVMSG)
412	ERR_NOTEXTTOSEND	:No text to send
421	ERR_UNKNOWNCOMMAND	<command> :Unknown command
422	ERR_NOMOTD	:MOTD File is missing
431	ERR_NONICKNAMEGIVEN	:No nickname given
432	ERR_ERRONEUSNICKNAME	<nick> :Erroneous nickname
433	ERR_NICKNAMEINUSE	<nick> :Nickname is already in use
441	ERR_USERNOTINCHANNEL	<nick> <channel> :They aren't on that channel
442	ERR_NOTONCHANNEL	<channel> :You're not on that channel
443	ERR_USERONCHANNEL	<nick> <channel> :is already on channel
451	ERR_NOTREGISTERED	:You have not registered
461	ERR_NEEDMOREPARAMS	<command> :Not enough parameters
462	ERR_ALREADYREGISTERED	:Unauthorized command (already registered)
464	ERR_PASSWDMISMATCH	:Password incorrect
471	ERR_CHANNELISFULL	<channel> :Cannot join channel (+l)
472	ERR_UNKNOWNMODE	<char> :is unknown mode char to me for <channel>
473	ERR_INVITEONLYCHAN	<channel> :Cannot join channel (+i)
475	ERR_BADCHANNELKEY	<channel> :Cannot join channel (+k)
482	ERR_CHANOPRIVSNEEDED	<channel> :You're not channel operator

ERR_ALREADYREGISTEREDはRFCのSymbol spellingに合わせる。

9. Numeric定数

```
namespace Numeric
{
    const int RPL_WELCOME = 1;
    const int RPL_YOURHOST = 2;
    const int RPL_CREATED = 3;
    const int RPL_MYINFO = 4;

    const int RPL_CHANNELMODEIS = 324;
    const int RPL_NOTOPIC = 331;
    const int RPL_TOPIC = 332;
    const int RPL_INVITING = 341;
    const int RPL_NAMREPLY = 353;
    const int RPL_ENDOFNAMES = 366;

    const int ERR_NOSUCHNICK = 401;
    const int ERR_NOSUCHCHANNEL = 403;
    const int ERR_CANNOTSENDOCHAN = 404;
    const int ERR_NOORIGIN = 409;
    const int ERR_NORECIPIENT = 411;
    const int ERR_NOTEXTTOSEND = 412;
    const int ERR_UNKNOWNCOMMAND = 421;
    const int ERR_NOMOTD = 422;
    const int ERR_NONICKNAMEGIVEN = 431;
    const int ERR_ERRONEUSNICKNAME = 432;
    const int ERR_NICKNAMEINUSE = 433;
    const int ERR_USERNOTINCHANNEL = 441;
    const int ERR_NOTONCHANNEL = 442;
    const int ERR_USERONCHANNEL = 443;
    const int ERR_NOTREGISTERED = 451;
    const int ERR_NEEDMOREPARAMS = 461;
    const int ERR_ALREADYREGISTERED = 462;
    const int ERR_PASSWDMismatch = 464;
    const int ERR_CHANNELISFULL = 471;
    const int ERR_UNKNOWNMODE = 472;
    const int ERR_INVITEONLYCHAN = 473;
    const int ERR_BADCHANNELKEY = 475;
    const int ERR_CHANOPRIVSNEEDED = 482;
}
```

Headerではconst intをnamespace scopeに置く。C++98では内部linkageになるため、ODR問題を避けられる。

10. Error Reply Helper

Server内の共通関数:

```
void sendNumeric(  
    int fd,  
    int code,  
    const std::string &parameters  
)
```

処理:

1. Client存在確認
2. Reply::numeric()を呼ぶ
3. queueToClient()へ渡す

頻出Helper:

```
void sendNeedMoreParams(int fd,  
                        const std::string &command);  
void sendNoSuchChannel(int fd,  
                      const std::string &channel);  
void sendNoSuchNick(int fd,  
                   const std::string &nickname);
```

Handler内でNumeric文字列を直接連結しない。

11. CommandとErrorの対応

Command	主なError
PASS	461, 462, 464
NICK	431, 432, 433
USER	461, 462
JOIN	403, 461, 471, 473, 475
PRIVMSG	401, 403, 404, 411, 412
KICK	401, 403, 441, 442, 461, 482
INVITE	401, 403, 442, 443, 461, 482
TOPIC	403, 442, 461, 482
MODE	401, 403, 441, 442, 461, 472, 482
PART	403, 442, 461
PING	409
Unknown	421

12. 入力Validation Error

Validationは次の3段階へ分ける。

12.1 Message構文

Parserが担当:

- ・ 空行
- ・ Commandなし
- ・ Parameter数16以上
- ・ NULを含む行

空行は無視する。Parameter数超過は先頭15個だけを使用せず、Message全体を不正として破棄する。

12.2 Command共通

Dispatcherまたは共通Helperが担当:

- ・ 登録状態
- ・ 必須Parameter数
- ・ 未知Command

12.3 Command固有

Handlerが担当:

- ・ Nickname形式
- ・ Channel名形式
- ・ Limit数値
- ・ Key形式
- ・ 対象存在確認
- ・ MemberとOperator権限

13. 512 byte制限

受信:

- ・ CRLF込み512 byteを超える行は切断対象
- ・ LFのみで届いた場合も、仮想CRLFを含めて512 byte以内と判断する

送信:

- ・ Reply生成後、CRLF追加前に510 byte以下を確認する
- ・ Numeric ReplyはParameterを切り詰めない
- ・ Names Replyだけは複数行へ分割する
- ・ PRIVMSG本文が送信行上限を超える場合は末尾を切らず、送信者へ412ではなく404を返して配送しない

内部Helper:

```
bool IrcUtil::fitsIrcLine(  
    const std::string &withoutCrlf  
)
```

14. system call Error分類

14.1 Server致命的

- ・ listen socketのsocket()失敗
- ・ listen socketのsetsockopt()失敗
- ・ listen socketのfcntl()失敗
- ・ bind()失敗
- ・ listen()失敗
- ・ listen FDのPOLLERR、POLLHUP、POLLNVAL
- ・ 停止signal以外によるpoll()失敗

処理:

1. 標準エラーへ概要を出力
2. 全FDを閉じる
3. 終了コード1で終了

14.2 接続単位

- ・ accepted FDのfcntl()失敗
- ・ Client FDのrecv()失敗
- ・ Client FDのsend()失敗
- ・ Client FDのpoll error
- ・ Buffer上限超過

処理:

- ・ 対象Clientだけを切断
- ・ Serverと他Clientは継続

14.3 回復可能

- ・ accept()失敗

処理:

- ・ Clientを追加せずイベントループ継続
- ・ 同じready通知でacceptを再実行しない

15. 内部Errorと不変条件違反

例:

- ・ Channel Member FDがClient Mapに存在しない
- ・ ClientのjoinedChannelsに存在するChannelがChannel Mapにない
- ・ Nickname索引が別FDを指す
- ・ pollfdにClient MapにないFDがある

Release動作:

1. 標準エラーへ記録
2. 操作対象がClientなら安全に切断予約
3. 存在しないIDは集合から削除
4. Server全体を可能な限り継続

Debug動作:

- ・ checkInvariants()を呼び、失敗箇所を明示する

例外をCommand入力エラーの制御フローに使用しない。

16. Buffer Error

Receive Buffer:

```
ERROR :Receive buffer limit exceeded
```

Send Buffer:

- ・ ERRORを追加するとさらにBufferが増えるため、通知せず切断する

Line length:

```
ERROR :Input line too long
```

ERRORはNumericではないServer Commandである。

```
:ircserv.local ERROR :Input line too long
```

可能ならqueueするが、即時切断で未送信になることを許容する。

17. Reply生成の安全性

Reply関数では次を保証する。

- ・ CR、LF、NULをParameterへ混入させない
- ・ Client入力由来のTrailingにCRまたはLFがあれば除去する
- ・ Prefixへspaceを含めない
- ・ codeが0から999の範囲外なら内部Errorとして空文字を返す
- ・ 空文字Replyをqueueしない

Sanitize Helper:

```
std::string IrcUtil::sanitizeMessageText(  
    const std::string &value  
)
```

CR、LF、NULを除去し、それ以外の文字は保持する。

18. Error発生時の状態変更規則

原則:

```
Validate
-> Resolve targets
-> Authorize
-> Mutate
-> Notify
```

検証、検索、権限確認のいずれかで失敗した場合、状態変更を行わない。

複数対象Command:

- ・ JOIN、PART、PRIVMSGの各対象は独立transactionとして扱う
- ・ 先の対象が成功し、後の対象が失敗しても先の成功を戻さない

MODE:

- ・ 各Mode文字を独立transactionとして扱う
- ・ 成功変更は維持し、失敗文字だけErrorを返す

図の元データ:

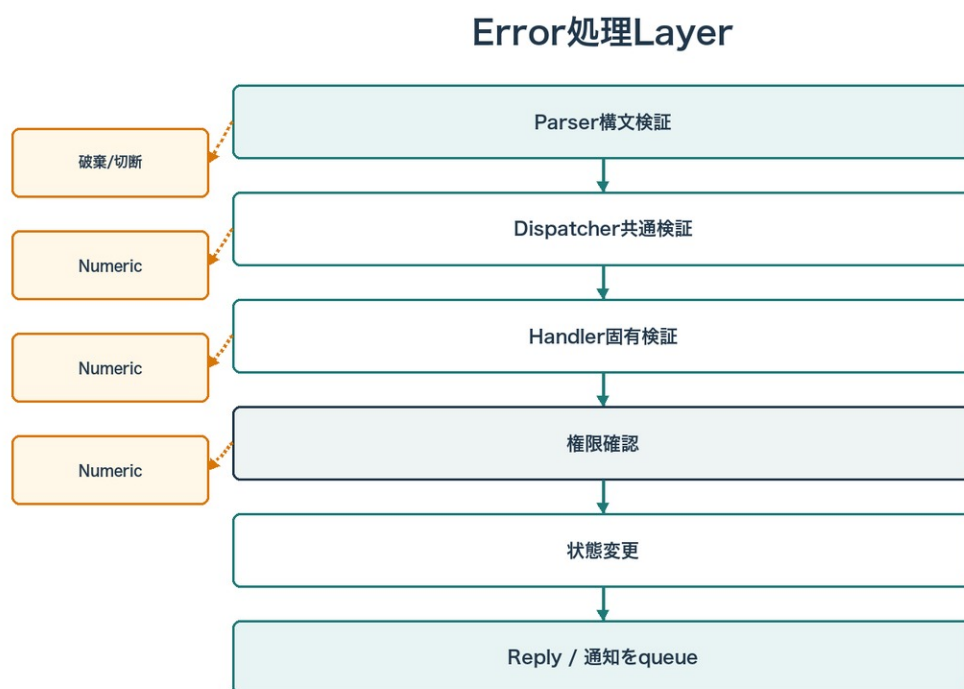


図1. Error処理Layer - 元データ: diagrams/error_handling_layers_ja.mmd

19. Replyテスト表

入力	期待先頭
PASSなしでJOIN	:ircserv.local 451 *
NICK	:ircserv.local 431 *
重複NICK	:ircserv.local 433 <current>
存在しないChannelへPART	:ircserv.local 403 <nick>
非OperatorがKICK	:ircserv.local 482 <nick>
JOIN成功	JOIN通知、331/332、353、366
MODE照会	:ircserv.local 324 <nick>
PING token	:ircserv.local PONG

20. 参照資料

- ・ ft_irc subject Version 11.0
- ・ RFC 2812, Internet Relay Chat: Client Protocol
- ・ <https://www.rfc-editor.org/rfc/rfc2812.html>

設計ではRFC 2812のMessage形式、512 byte制限、主要CommandとNumeric Replyを基準にする。課題要件と衝突する場合はft_irc subjectを優先する。

21. 実装完了条件

- ・ すべてのNumeric Replyが3桁と正しいtargetを持つ
- ・ Reply生成がHandlerから分離されている
- ・ 送信行がCRLFで終わる
- ・ Names Replyが必要に応じて複数行へ分割される
- ・ 未登録Clientへのtargetが*になる
- ・ 登録完了時に001から004と422が順番に送信される
- ・ 入力Errorで部分的な状態変更が残らない
- ・ 接続単位のI/O ErrorでServer全体が停止しない
- ・ 致命的Errorで全FDを閉じて終了する
- ・ Error Reply自体が512 byte制限を超えない
- ・ 不正入力にCRLF injectionが含まれても追加Commandとして送信されない